



POLITECNICO DI BARI

DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELL'INFORMAZIONE

LAUREA MAGISTRALE IN INGEGNERIA DELL'AUTOMAZIONE

Tesina in
Dynamical Systems Theory

Obstacle avoidance with Ned2 manipulator using CHOMP optimizer

Professori:

Prof. Ing. Mariagrazia Dotoli
Ing. Raffaele Carli
Ing. Fabio Mastromarino

Studenti:

Angelo Gregucci
Leonardo Li Bergolis

ANNO ACCADEMICO 2024-2025

Indice

1	Introduzione	1
1.1	Stato dell'arte	1
1.2	Task di pick and place	2
1.3	Spazio dei Giunti e Spazio delle Fasi	3
1.4	Manipolatore utilizzato: Niryo Ned2	3
2	CHOMP	5
2.1	Formulazione	5
2.1.1	Gradiente	6
2.1.2	Convergenza	7
2.1.3	Rappresentazioni ambiente	7
2.2	Confronto con RRT	7
2.3	Applicazione al caso di studio	7
3	Analisi dello spazio operativo	9
3.1	Ambiente di simulazione	9
3.2	Analisi punti raggiungibili	9
3.3	Indice scelto	11
4	Simulazioni e analisi dei risultati	12
4.1	Traiettoria Ottima	12
4.1.1	Setup	12
4.1.2	Risultati	12
4.2	Limiti del CHOMP	14
5	Conclusione e Sviluppi Futuri	16
	Riferimenti	18
A	Codici Matlab	19
A.1	ned2 import	19
A.2	Matlab Code	21
A.2.1	Init	21
A.2.2	Main	21
A.2.3	wksp analysis	23

Sommario

Il progetto presentato si basa sull'impiego di un manipolatore, in particolare il **Nyrio Ned2**, per l'esecuzione di operazioni di *pick and place* in ambiente **Matlab**. La traiettoria ottimale dell'oggetto viene determinata mediante la funzione **optimize** dell'algoritmo **CHOMP**, che permette l'*obstacle avoidance* rispetto agli ostacoli presenti nello scenario simulato. Tali ostacoli vengono rappresentati in forma approssimata tramite sfere, da fornire in input all'algoritmo. Dopo aver definito il *workspace* e il modello del manipolatore, viene condotta un'analisi mirata a valutare le prestazioni e i limiti della simulazione in presenza di ostacoli, con particolare attenzione all'impatto dell'approssimazione geometrica adottata da CHOMP. L'obiettivo è esaminare come varia il comportamento del sistema al modificarsi dell'ambiente di lavoro.

Capitolo 1

Introduzione

Nel contesto della robotica industriale e collaborativa, i **manipolatori** robotici rappresentano una delle tecnologie fondamentali per l'automazione di operazioni complesse e ripetitive. Si tratta di dispositivi meccanici programmabili progettati per eseguire movimenti precisi e controllati nello spazio tridimensionale, emulando in parte la destrezza del braccio umano. Questi sistemi sono composti da una catena di *link* rigidi connessi tramite *giunti* mobili (rotazionali o prismatici), e sono in grado di posizionare e orientare un effettore finale (*end-effector*) in una configurazione desiderata, mediante il controllo di uno o più gradi di libertà, noti come *Degrees of Freedom* (**DoF**). I manipolatori trovano applicazione in una vasta gamma di settori, tra cui:

- **Industria manifatturiera:** utilizzati in catene di montaggio per operazioni di saldatura, pittura, imballaggio, pallettizzazione e assemblaggio di componenti.
- **Automotive:** impiegati per l'assemblaggio di motori, la saldatura della carrozzeria e l'installazione di componenti con alta precisione e velocità.
- **Chirurgia robotica:** manipolatori miniaturizzati assistono i chirurghi in interventi delicati, offrendo movimenti estremamente precisi e meno invasivi.
- **Esplorazione spaziale e subacquea:** utilizzati su veicoli robotici per manipolare oggetti in ambienti estremi dove l'intervento umano diretto non è possibile.
- **Logistica e magazzini automatizzati:** impiegati per prelevare e posizionare prodotti in scaffalature (nel progetto simuleremo proprio questo impiego), migliorando l'efficienza e riducendo l'errore umano.
- **Laboratori e ambienti pericolosi:** permettono di manipolare sostanze chimiche o materiali radioattivi in sicurezza.

Tra le operazioni più comuni affidate ai manipolatori vi sono i compiti di *pick and place*, che consistono nel prelevare un oggetto da una posizione iniziale e riposizionarlo con precisione in una destinazione predefinita. Tali attività, essenziali nei processi di automazione, packaging e assemblaggio, richiedono una serie di prerequisiti tecnici per poter essere eseguite in modo efficace. In particolare, è necessaria una pianificazione accurata del movimento del manipolatore, un controllo preciso dell'end-effector, una gestione efficiente della cinematica diretta e inversa, nonché la capacità di evitare collisioni con l'ambiente circostante.

1.1 Stato dell'arte

Il problema della pianificazione del moto per manipolatori robotici, in particolare per compiti di *pick and place*, è stato affrontato con numerosi approcci che differiscono per efficienza computazionale,

capacità di gestione degli ostacoli, regolarità della traiettoria e adattabilità all'ambiente. Tra le tecniche più rilevanti si annoverano:

- **CHOMP** (Covariant Hamiltonian Optimization for Motion Planning) [1]: è un metodo di ottimizzazione del moto che sfrutta il gradiente della traiettoria rispetto a una funzione costo composta da termini di regolarità e penalizzazione degli ostacoli. È particolarmente efficace nel generare traiettorie lisce evitando collisioni, pur richiedendo una buona inizializzazione e mappe signed-distance.
- **STOMP** (Stochastic Trajectory Optimization for Motion Planning) [2]: è un metodo basato su campionamento stocastico che non necessita del calcolo del gradiente del costo rispetto alla traiettoria. Questo lo rende adatto in scenari dove la funzione costo è non differenziabile, a scapito però dell'efficienza in ambienti complessi.
- **TrajOpt** (Trajectory Optimization) [3]: riformula la pianificazione del moto come un problema di ottimizzazione convessa sequenziale. Si distingue per la capacità di generare rapidamente traiettorie localmente ottime, anche in ambienti densi di ostacoli, utilizzando vincoli espliciti di sicurezza e dinamica.
- **RRT e sue varianti**: i metodi sampling-based, come l'RRT migliorato in [4], sono spesso utilizzati per trovare rapidamente traiettorie valide anche in ambienti molto complessi. Tuttavia, le traiettorie generate sono spesso sub-ottimali e richiedono un successivo affinamento per migliorarne regolarità e sicurezza.

Questi approcci possono essere distinti in due macro-categorie:

- **Metodi di campionamento (Sampling-based)**: come RRT e PRM, garantiscono copertura dello spazio ma generano traiettorie non ottimali.
- **Metodi di ottimizzazione (Optimization-based)**: come CHOMP, STOMP e TrajOpt, si focalizzano sul miglioramento locale di una traiettoria iniziale, fornendo soluzioni più fluide e sicure.

La scelta del metodo più adatto dipende dal contesto applicativo: CHOMP, ad esempio, è particolarmente indicato quando si dispone di una buona inizializzazione e si richiede alta qualità della traiettoria in termini di smoothness e sicurezza.

1.2 Task di pick and place

Per eseguire un'operazione di pick and place in modo efficace, un manipolatore deve soddisfare i seguenti requisiti dal punto di vista cinematico:

- **Numero sufficiente di gradi di libertà (DoF)**: Almeno 6 DoF per garantire piena controllabilità della posizione e dell'orientamento dell'end-effector nello spazio tridimensionale.
- **Risoluzione della cinematica diretta**: Capacità di calcolare la posizione e l'orientamento dell'end-effector dati i valori delle giunzioni articolari.
- **Risoluzione della cinematica inversa**: Capacità di calcolare una o più configurazioni articolari che consentano di raggiungere una data posa dell'end-effector.
- **Continuità e fluidità delle traiettorie**: Le traiettorie devono essere prive di discontinuità angolari o geometriche per evitare movimenti bruschi o instabili.
- **Rispetto dei limiti articolari**: Ogni giunto deve operare entro i propri limiti di posizione, velocità e accelerazione per garantire sicurezza e fattibilità.

La pianificazione del moto, in questo contesto, assume un ruolo centrale. In scenari reali, caratterizzati dalla presenza di ostacoli, spazi ristretti o vincoli cinematici severi, il problema diventa computazionalmente complesso. Di conseguenza, la scelta dell'algoritmo di pianificazione influenza in modo significativo le prestazioni del sistema. Per il presente progetto, si è scelto di adottare l'algoritmo **CHOMP** (*Covariant Hamiltonian Optimization for Motion Planning*), una soluzione basata su tecniche di ottimizzazione del gradiente, che verrà analizzata in dettaglio nel Capitolo 2. CHOMP si distingue per la sua capacità di generare traiettorie fluide ed energeticamente efficienti, garantendo al contempo di evitare gli ostacoli, risultando particolarmente adatto per applicazioni dinamiche di pick and place in ambienti semi-strutturati.

1.3 Spazio dei Giunti e Spazio delle Fasi

Nel contesto della robotica manipolativa, è fondamentale distinguere tra **spazio dei giunti** (*joint space*) e **spazio delle fasi** (*phase space*), poiché ciascuno rappresenta una diversa modalità di descrizione dello stato del robot.

Lo *spazio dei giunti* è definito come l'insieme di tutte le possibili configurazioni dei giunti di un robot. Ogni configurazione è rappresentata da un vettore:

$$\mathbf{q} = [q_1, q_2, \dots, q_n]^T$$

dove q_i rappresenta la posizione del giunto i -esimo, che può essere un angolo (per giunti rotazionali) o uno spostamento lineare (per giunti prismatici), n equivale al numero di giunti (e quindi al numero dei g.d.l.), quindi per questo manipolatore avremo vettori con 6 elementi. Questo spazio viene comunemente utilizzato per descrivere la postura del robot e per pianificare traiettorie in ambienti complessi, evitando collisioni attraverso la manipolazione diretta delle configurazioni dei giunti.

Lo *spazio delle fasi*, invece, estende lo spazio dei giunti includendo anche le velocità associate a ciascun giunto:

$$\mathbf{x} = [\mathbf{q}, \dot{\mathbf{q}}]^T = [q_1, q_2, \dots, q_n, \dot{q}_1, \dot{q}_2, \dots, \dot{q}_n]^T$$

In questo spazio, ogni stato del robot è descritto non solo dalla sua configurazione, ma anche dal modo in cui essa varia nel tempo. Questa rappresentazione è particolarmente utile per l'analisi della dinamica, la progettazione di controllori e lo studio della stabilità del moto.

In sintesi, mentre lo spazio dei giunti è tipicamente utilizzato per la pianificazione della traiettoria e per la risoluzione della cinematica inversa, ragione per cui si è scelto di utilizzare questo approccio nel progetto, lo spazio delle fasi è essenziale per modellare il comportamento dinamico del robot e per implementare strategie di controllo avanzate.

1.4 Manipolatore utilizzato: Niryo Ned2

In questo progetto è stato utilizzato il manipolatore **Niryo Ned2**, in Figura 1.2, un manipolatore a 6 gradi di libertà ispirato dal mondo industriale ma progettato per scopi didattici e di ricerca. Il manipolatore presenta dimensioni pari a 181,60 mm × 187,60 mm × 436,22 mm.



Figura 1.1: Custom Gripper



Figura 1.2: Niryo Ned2

Le specifiche tecniche principali del manipolatore utilizzato sono [5]:

- **Carico utile (Payload):** 300 g
- **Precisione (Accuracy):** ± 0.5 mm
- **Raggio d'azione (Reach):** 49 cm / $\varnothing$ 1 m
- **Peso:** 7 kg

A seconda dell'applicazione è possibile scegliere il gripper più adatto, per questo progetto è stato utilizzato il custom gripper [6] mostrato in figura 1.1.

Capitolo 2

CHOMP

Il *Covariant Hamiltonian Optimization for Motion Planning* (**CHOMP**) [1] è un algoritmo di ottimizzazione per la pianificazione del moto, progettato per generare traiettorie fluide ed efficienti dal punto di vista dinamico, attraverso l'impiego di tecniche di discesa del gradiente. È ampiamente adottato in ambito robotico, in particolare per compiti di *obstacle avoidance*, dove è necessario generare traiettorie ottimali che rispettino vincoli cinematici ed evitino collisioni con gli ostacoli presenti nell'ambiente.

2.1 Formulazione

CHOMP minimizza una funzione obiettivo composta da due termini principali: un termine di **regolarità** (*smoothness*) e uno di **collisione**. Il primo penalizza traiettorie con brusche variazioni di velocità o accelerazione, promuovendo quindi movimenti continui e fluidi; il secondo introduce un potenziale repulsivo in prossimità degli ostacoli, incentivando l'esplorazione di traiettorie sicure.

Il problema di pianificazione può essere formalizzato come segue:

$$\mathcal{U}[\xi] = \mathcal{F}_{\text{smooth}}[\xi] + \lambda \mathcal{F}_{\text{obs}}[\xi]$$

dove λ è un parametro di ponderazione tra i due termini. Più precisamente:

- $\mathcal{F}_{\text{smooth}}[\xi]$ è un funzionale definito su una metrica nello spazio delle traiettorie, e può includere derivate di ordine superiore (come accelerazione o jerk) per modellare proprietà cinematiche desiderabili;
- $\mathcal{F}_{\text{obs}}[\xi]$ rappresenta un integrale del costo indotto dalla prossimità agli ostacoli lungo la traiettoria, espresso tramite un campo di costo definito nello spazio di lavoro.

Questa formulazione consente di affrontare in maniera efficace uno dei principali trade-off nella pianificazione del moto: generare traiettorie che coniughino sicurezza e fluidità.

Poiché lo spazio di configurazione di un manipolatore robotico ha dimensione elevata, è utile semplificare il problema proiettando il campo di costo direttamente nello spazio di lavoro (2D o 3D). Questo consente di definire le funzioni obiettivo tramite operazioni analitiche più semplici, come il calcolo del gradiente spaziale.

Sia quindi una traiettoria generica $\xi : [0, 1] \rightarrow C \subset \mathbb{R}^d$, dove C è lo spazio di configurazione del robot, e siano assegnate le configurazioni iniziale e finale:

$$\xi(0) = q_0, \quad \xi(1) = q_1$$

Il termine di regolarità viene espresso come:

$$\mathcal{F}_{\text{smooth}}[\xi] = \frac{1}{2} \int_0^1 \left\| \frac{d\xi(t)}{dt} \right\|^2 dt \quad (2.1)$$

e può essere esteso includendo derivate superiori, ad esempio:

$$\mathcal{F}_{\text{smooth}}[\xi] = \frac{1}{2} \int_0^1 \left\| \frac{d^n \xi(t)}{dt^n} \right\|^2 dt$$

per modellare ulteriori requisiti sulla continuità.

Il termine di collisione tiene invece conto della distanza tra il corpo del robot e gli ostacoli nello spazio di lavoro. Sia $\mathcal{B} \subset \mathbb{R}^3$ l'insieme dei punti del corpo del robot, e sia $x : C \times \mathcal{B} \rightarrow \mathbb{R}^3$ la funzione di cinematica diretta che mappa la configurazione $q \in C$ e un punto del corpo $u \in \mathcal{B}$ nella posizione spaziale $x(q, u)$. Definito un campo di costo $c : \mathbb{R}^3 \rightarrow \mathbb{R}$ che cresce in prossimità degli ostacoli, il termine di collisione può essere espresso come:

$$\mathcal{F}_{\text{obs}}[\xi] = \frac{1}{2} \int_0^1 \int_{\mathcal{B}} c(x(\xi(t), u)) \left\| \frac{d\xi(t)}{dt} \right\| du dt \quad (2.2)$$

Questa formulazione è invariante rispetto alla parametrizzazione temporale della traiettoria, poiché il campo di costo viene ponderato rispetto alla lunghezza d'arco, ovvero alla norma della velocità. In tal modo, la traiettoria ottimizzata mantiene le sue caratteristiche anche al variare della velocità di percorrenza.

2.1.1 Gradiente

I metodi con gradiente sono tipici per l'ottimizzazione di queste funzioni oggetto in quanto contengono informazioni circa la forma delle funzioni oggetto. Sia $\nabla \mathcal{U}$ la perturbazione $\phi : [0, 1] \mapsto \mathbb{R}^d$ che massimizza $\nabla \mathcal{U}[\xi + \epsilon \phi]$ come $\epsilon \rightarrow 0$, ora il funzionale diviene $\mathcal{U}[\xi] = \int v(\xi(t), \xi'(t)) dt$ e considerando la perturbazione ϕ applicata a ξ di una quantità ϵ per arrivare alla nuova funzione $\tilde{\xi} = \xi + \epsilon \phi$ e con ξ e ϕ costanti, avremo $\mathcal{U}[\tilde{\xi}]$ come una funzione $f(\epsilon)$ con

$$\frac{df}{d\epsilon} = \int \left(\frac{\partial v}{\partial \tilde{\xi}} \cdot \frac{\partial \tilde{\xi}}{\partial \epsilon} + \frac{\partial v}{\partial \tilde{\xi}'} \cdot \frac{\partial \tilde{\xi}'}{\partial \epsilon} \right) dt \quad (2.3)$$

A seguito di numerose semplificazioni possiamo definire il gradiente funzionale come

$$\nabla \mathcal{U}[\xi] = \frac{\partial v}{\partial \xi} - \frac{d}{dt} \frac{\partial v}{\partial \xi'} \quad (2.4)$$

il quale può incorporare anche derivate di ordine superiore. Inoltre il gradiente sparisce per alcuni punti critici di $\mathcal{U}$ i quali contengono il minimo che si sta cercando, ovvero $\nabla \mathcal{U} = 0$, quindi possiamo trovare la convergenza più ripida del funzionale eseguendo passi in modo iterativo verso la direzione negativa del gradiente funzionale fin quando non raggiungiamo un minimo locale

$$\xi_{i+1} = \xi_i - \mu_i \nabla \mathcal{U}[\xi] \quad (2.5)$$

Infine poichè la funzione oggetto è la somma di due termini sulla smoothness e gli ostacoli, il gradiente funzionale della smoothness è

$$\nabla \mathcal{F}_{\text{smooth}}[\xi](t) = -\frac{d^2}{dt^2} \xi(t) \quad (2.6)$$

mentre per gli ostacoli è

$$\nabla \mathcal{F}_{\text{obs}}[\xi](t) = \int_{\mathcal{B}} J^T \|x'\| [(I - \hat{x}' \hat{x}'^T) \nabla c - ck] du \quad (2.7)$$

con $k = \|x'\|^{-2} (I - \hat{x}' \hat{x}'^T) x''$ il vettore curvatura lungo la traiettoria nello spazio di lavoro generata da uno specifico punto del corpo, x' e x'' sono la velocità e l'accelerazione del punto del corpo, $\hat{x}'$ è il vettore velocità normalizzato e J è il Jacobiano che descrive la cinematica del punto.

2.1.2 Convergenza

Per trovare una iterazione che parte da ξ_0 , la quale è la traiettoria iniziale e arriva a ξ_{i+1} traiettoria ridefinita e ottimizzata passando per ξ_i la traiettoria del passo precedente, si parte cercando di massimizzare la discesa della funzione oggetto mantenendo una differenza tra ξ_{i+1} e ξ_i piccola. Si linearizza $\mathcal{U}$ attorno a ξ con la serie di Taylor al primo ordine

$$\mathcal{U}[\xi] \approx \mathcal{U}[\xi_i] + (\xi - \xi_i)^T \nabla \mathcal{U}[\xi_i] \quad (2.8)$$

il problema di ottimizzazione ora diventa

$$\xi_{i+1} = \operatorname{argmin}(\mathcal{U}[\xi_i]) + (\xi - \xi_i)^T \nabla \mathcal{U}[\xi_i] + \frac{\mu}{2} \|\xi - \xi_i\|_M^2 \quad (2.9)$$

con il termine $\|\xi - \xi_i\|_M^2 = (\xi - \xi_i)^T M (\xi - \xi_i)$ la norma al quadrato della variazione della traiettoria rispetto al tensore metrico M e μ un coefficiente di regolazione tra minimizzare $\mathcal{U}$ e il passo, si definisce $M = A$. Infine differenziando i termini a destra rispetto ξ e ponendola a 0 arriviamo alla regola iterativa:

$$\xi_{i+1} = \xi_i - \frac{1}{\mu} A^{-1} \nabla \mathcal{U}[\xi_i] \quad (2.10)$$

2.1.3 Rappresentazioni ambiente

Il set $\mathcal{B}$ dei punti del corpo del robot può essere approssimato con delle primitive geometriche le quali includono il corpo del robot come sfere, capsule e poliedri. Una traiettoria ξ è *collision-free* se per tutte le configurazioni $q_i \in \xi$ per ogni punto $x_u \in x(q_i, u)$ la distanza tra quel punto e l'ostacolo è maggiore di una soglia $\epsilon > 0$. Sia $D : R^w \rightarrow R$ una funzione che fornisce la distanza tra qualsiasi punto dello spazio di lavoro e la superficie dell'ostacolo più vicino, questa funzione può essere implementata con delle primitive geometriche a scelta come sfere, cilindri e box[1].

2.2 Confronto con RRT

Un secondo algoritmo anch'esso molto diffuso è Il Rapidly-exploring Random Tree (RRT) ma ha delle differenze sostanziali rispetto al CHOMP.

E' un algoritmo basato sul campionamento, mentre il CHOMP è di tipo esplorativo: costruisce prima una traiettoria grossolana da cui partire per poi ottimizzarla sempre di più fino a soddisfare la funzione di costo puntando alla fluidità di movimento, l'RRT invece costruisce un albero esplorando lo spazio di configurazioni fino a trovare il percorso migliore per la traiettoria dal punto iniziale a quello finale. Tende maggiormente a trovare una soluzione rapida mentre il CHOMP si focalizza più sulla qualità del movimento.

In definitiva l'RRT è usato più per l'analisi di traiettorie rapide in ambienti di lavoro complessi e dinamici in cui ci si focalizza sulla velocità di processo più che sulla fluidità del movimento, invece il CHOMP è più adatto in ambienti di lavoro meno complessi e soprattutto statici, dove ciò che conta è proprio la qualità del movimento del manipolatore.

2.3 Applicazione al caso di studio

Per generare una traiettoria utile è necessario dapprima generare l'ambiente di simulazione del robot, al cui interno sono presenti tutti gli ostacoli che esso deve evitare. Un esempio pratico è rappresentato dal caso di studio considerato: è stato realizzato un ambiente in cui alcuni muri fungono da ostacoli che il manipolatore deve evitare per eseguire l'operazione di *Pick and Place* di un piccolo cilindro.

Dal punto di vista applicativo, l'algoritmo CHOMP approssima tutti gli oggetti importati nel proprio ambiente tramite sfere o cilindri a raggio variabile. La risoluzione dell'approssimazione viene impostata tramite un apposito comando, in base al quale l'algoritmo genera queste figure geometriche e le distribuisce all'interno del volume dell'oggetto, riempiendolo completamente. Naturalmente, al variare della risoluzione impostata, cambia anche il livello di precisione con cui viene rappresentata la geometria degli oggetti. Dal punto di vista geometrico, per quanto le sfere e i cilindri utilizzati possano essere di dimensioni ridotte, essi tendono comunque a sovrastimare il volume reale dell'oggetto rappresentato.

E' stato importato il manipolatore tramite la funzione *ned2import*, generando una configurazione iniziale; successivamente è stata passata questa all'algoritmo, approssimando così l'intero manipolatore con delle sfere tramite il comando *manipulatorCHOMP()* (A.2.1). Questi passaggi permettono di inizializzare il robot per creare successivamente le traiettorie.

Una volta definiti tutti gli ostacoli e gli oggetti da includere nell'ambiente di simulazione (A.2.2), è necessario crearli tramite gli appositi comandi. Nel caso in esame, le mura sono state generate utilizzando il comando *collisionBox()*, mentre il cilindro è stato creato mediante il comando *collisionCylinder()*, impostando in entrambi i casi le dimensioni e la posizione del centro in modo opportuno.

Per posizionare correttamente le mura e il cilindro all'interno dell'ambiente, è stato utilizzato il comando *trvec2tform*. Successivamente, per segnalare all'algoritmo CHOMP che le mura rappresentano ostacoli invalicabili per il manipolatore, è stato impiegato il comando *chomp.SphericalObstacle*, al quale viene passato l'ambiente precedentemente approssimato tramite sfere (A.2.2).

Capitolo 3

Analisi dello spazio operativo

3.1 Ambiente di simulazione

L'ambiente generato è composto da uno scaffale posizionato alla destra del manipolatore, sul quale viene eseguita l'operazione di *place* del cilindro. Lo scaffale è costituito da due piani, di cui il superiore presenta un'altezza variabile in funzione di un parametro b . Sulla sinistra del manipolatore è invece posizionato il cilindro destinato all'operazione di presa.

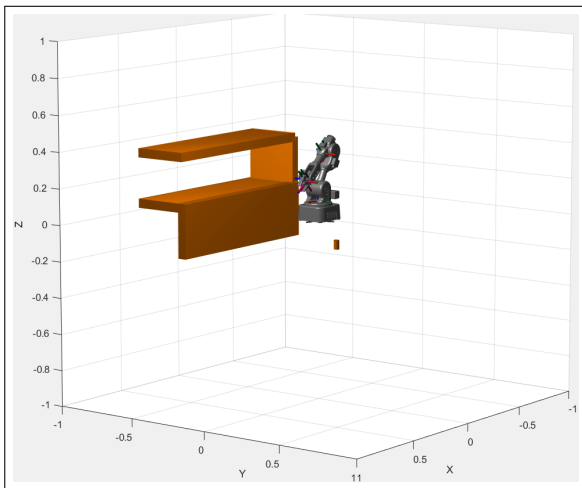


Figura 3.1: Ambiente di simulazione

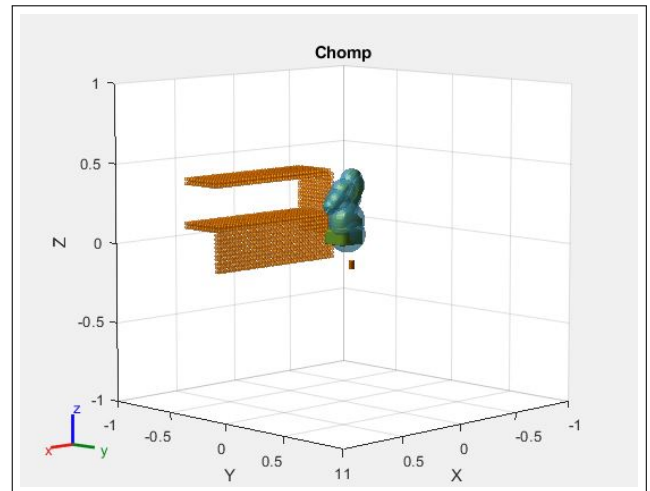


Figura 3.2: Ambiente modellato a sfere

E' stata scelta questa tipologia di ambiente con l'altezza parametrica dell'ostacolo superiore poiché l'obiettivo di questo progetto è quello di testare i limiti dell'algoritmo *CHOMP*; nel capitolo 4 la simulazione verrà divisa in due casi diversi.

3.2 Analisi punti raggiungibili

Il *workspace* di un manipolatore è definito come l'insieme di tutte le posizioni nello spazio tridimensionale che l'end-effector può raggiungere, tenendo conto delle limitazioni cinematiche, geometriche e articolari del robot. Si tratta di una caratteristica fondamentale, in quanto stabilisce i confini operativi entro cui il manipolatore può interagire con l'ambiente circostante.

Nel contesto della pianificazione del movimento, la conoscenza accurata del workspace è essenziale per diversi motivi. In primo luogo, consente di verificare la validità delle configurazioni iniziale e finale di una traiettoria, assicurandosi che entrambe rientrino nello spazio fisicamente accessibile dal robot.

Inoltre, l'analisi della distribuzione dei punti raggiungibili permette di individuare regioni dello spazio con diversa densità di raggiungibilità, utile per selezionare traiettorie che attraversino configurazioni vantaggiose e più stabili. Un altro aspetto rilevante è la prevenzione delle configurazioni singolari, ovvero quelle situazioni in cui il robot perde uno o più gradi di libertà effettivi. In prossimità di tali configurazioni, l'end-effector può risultare incapace di muoversi in determinate direzioni, oppure piccole variazioni nelle posizioni dei giunti possono causare grandi spostamenti dell'end-effector (o viceversa, i giunti si muovono senza che l'end-effector cambi posizione). Tali condizioni, oltre a compromettere l'accuratezza del controllo, possono introdurre instabilità numeriche e rischi operativi. Per questo motivo, l'analisi preventiva del workspace risulta cruciale nella progettazione di traiettorie sicure ed efficienti.

Dall'analisi svolta è stato calcolato un set dei punti raggiungibili; da questo, mostrato in Figura 3.3 sono stati estratti due punti, successivamente utilizzati come configurazione di *pick* (nello stesso punto è posizionato il cilindro) e di *place* per la traiettoria ottimizzata.

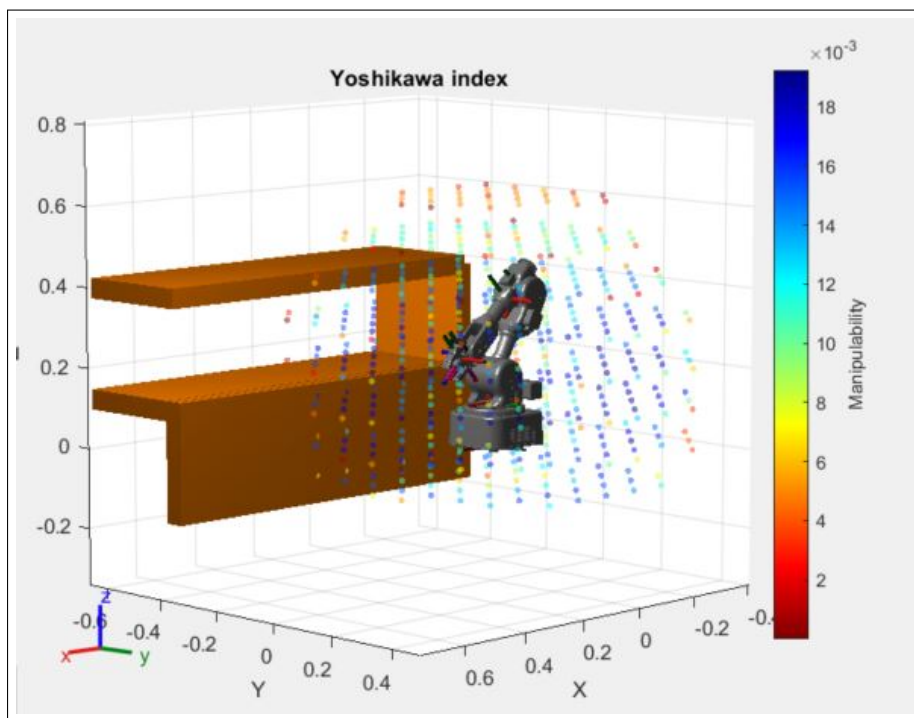


Figura 3.3: Yoshikawa index analysis

Per analizzare la qualità delle configurazioni raggiungibili è stata utilizzata una funzione built-in di MATLAB `manipulabilityIndex`, la quale supporta tre principali indici:

1. **Yoshikawa Index:**

$$M(q) = \sqrt{\det(J(q)J^T(q))}$$

dove $J(q)$ è la matrice Jacobiana nella configurazione articolare q . Questo indice misura il volume dell'ellissoide di velocità e fornisce una stima della destrezza locale del manipolatore. Valori elevati indicano maggiore capacità di movimento in tutte le direzioni. Risulta inoltre efficiente dal punto di vista computazionale.

2. **Inverse Condition Number:**

$$\text{ICN}(q) = \frac{\sigma_{\min}(J(q))}{\sigma_{\max}(J(q))}$$

esprime il rapporto tra i valori singolari minimo e massimo della matrice Jacobiana, fornendo una misura della "simmetria" del movimento in tutte le direzioni. Valori vicini a 1 indicano

un comportamento ben bilanciato del manipolatore, mentre valori prossimi a 0 segnalano una configurazione vicina alla singolarità. Questo indice è utile per valutare la stabilità numerica e l'affidabilità dei movimenti, ma non fornisce una misura diretta della capacità complessiva di movimento del robot.

3. Asada Index:

$$M_{\text{Asada}} = \frac{\lambda_{\min}(M)}{\lambda_{\max}(M)}$$

dove M è la matrice di inerzia operativa. Questo indice tiene conto della dinamica del sistema, valutando la capacità di accelerazione uniforme. È più adatto in contesti dinamici e di controllo avanzato.

3.3 Indice scelto

Nel contesto di questo progetto, è stato adottato lo **Yoshikawa Index** per la sua efficacia nel fornire una misura semplice e immediata della destrezza del manipolatore. L'obiettivo principale era pianificare una traiettoria ottimale all'interno di un ambiente statico, privilegiando la fluidità del movimento e la raggiungibilità dei punti, piuttosto che considerazioni di tipo dinamico o numerico.

Dopo l'analisi del workspace, in cui vengono identificati i punti raggiungibili nello spazio cartesiano (ovvero nello spazio operativo del robot), è necessario trasformare tali punti in coordinate di giunto. Questo perché il controllo effettivo del robot avviene nello *spazio dei giunti*: ogni movimento del robot è realizzato agendo sugli attuatori che controllano i singoli giunti, regolando angoli o traslazioni specifiche.

Sebbene un punto nello spazio operativo possa definire un obiettivo desiderato per l'end-effector, il manipolatore deve conoscere la configurazione dei propri giunti per raggiungerlo. Questa trasformazione avviene tramite la risoluzione della cinematica inversa $\mathbf{q} = \text{ik}(T_d)$, che fornisce le coordinate di giunto $\mathbf{q}$ associate a una data posa nello spazio cartesiano. T_d è la trasformazione omogenea che descrive la posa target dell'end-effector. Solo dopo questa trasformazione è possibile generare traiettorie, evitare ostacoli e controllare il robot lungo un percorso desiderato. A.2.3

Capitolo 4

Simulazioni e analisi dei risultati

4.1 Traiettorie Ottima

La generazione della traiettoria ottima nel contesto di questo progetto avviene tramite l'utilizzo della funzione `optimize` dell'oggetto `manipulatorCHOMP`, fornito dal Robotics System Toolbox di MATLAB [7]. Tale funzione consente di determinare una sequenza di configurazioni articolari che permetta al manipolatore di passare dalla posizione iniziale a quella di *pick*, per poi passare a quella di *place*, evitando eventuali collisioni e garantendo fluidità nel movimento. Nel caso di studio, il problema è suddiviso in due fasi distinte:

1. Traiettorie iniziale dall'home position alla configurazione di *pick*.
2. Traiettorie successiva dalla posizione di *pick* alla configurazione di *place*.

Entrambe le traiettorie sono state calcolate utilizzando il tipo `minjerkpolytraj` come interpolatore iniziale per la traiettoria, che garantisce una buona regolarità temporale e geometrica.

4.1.1 Setup

Come descritto nel Capitolo 2, CHOMP minimizza una funzione di costo che incorpora due componenti principali:

- una penalizzazione per l'irregolarità della traiettoria, ovvero un termine di *smoothness*, definito in MATLAB mediante le proprietà `CHOMP.SmoothnessOptions.SmoothnessCostWeight`;
- un potenziale di repulsione in prossimità degli ostacoli, per favorire l'*obstacle avoidance*, definito in MATLAB mediante le proprietà `CHOMP.CollisionOptions.CollisionCostWeight`.

I risultati sono influenzati da come viene tarato il `CHOMP.SolverOptions.LearningRate`. Durante la progettazione sono stati adottati i seguenti parametri di ottimizzazione:

- **Learning rate:** 5 – imposta la velocità di aggiornamento nel gradiente discendente.
- **Smoothness weight:** 1×10^{-3} – regola quanto è penalizzato un movimento irregolare.
- **Collision weight:** 1 – definisce quanto fortemente il robot evita le zone vicine agli ostacoli.

4.1.2 Risultati

La traiettoria risultante è una sequenza continua e liscia di configurazioni articolari, che attraversa lo spazio operativo evitando eventuali collisioni. In fase di simulazione, il manipolatore esegue un movimento privo di collisioni con ostacoli e rispetta i vincoli cinematici definiti nel modello (A.2.2).

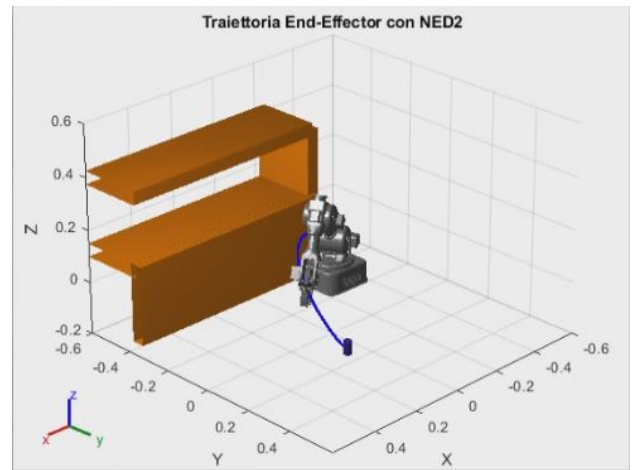
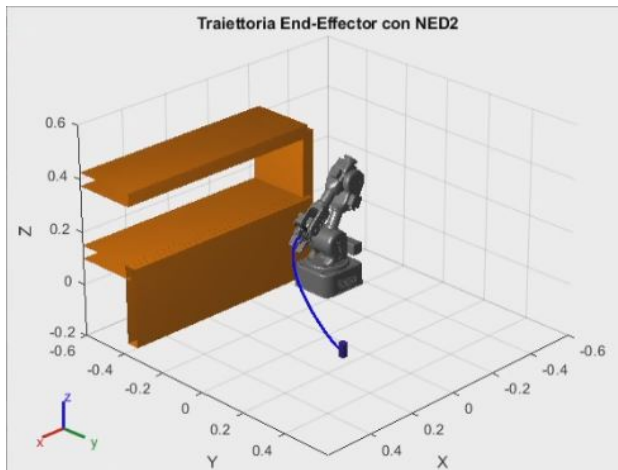


Figura 4.1: Manipolatore in fase di pick

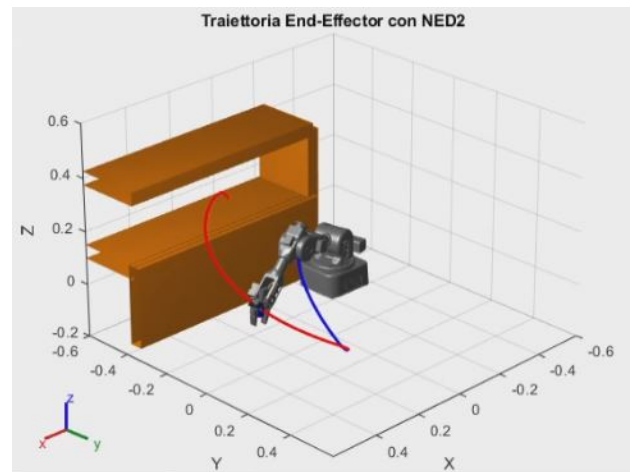
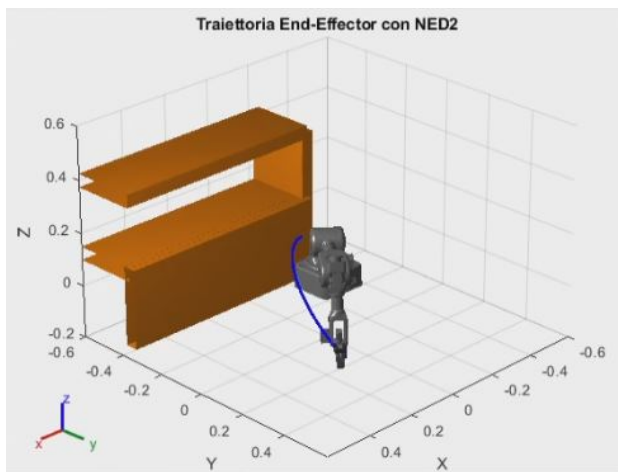


Figura 4.2: Manipolatore in fase di place

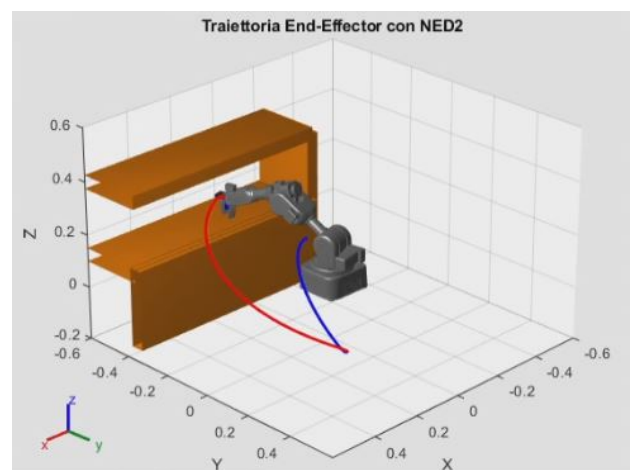


Figura 4.3: Manipolatore in fase di place

4.2 Limiti del CHOMP

Oltre alle limitazioni già evidenziate relative all'approssimazione geometrica degli ostacoli, in particolare l'uso di sfere che può sovrastimare i volumi reali, un ulteriore limite significativo di CHOMP riguarda la definizione della funzione di costo.

La funzione di costo utilizzata dall'algoritmo, infatti, non sempre rispecchia fedelmente le preferenze dell'utente né il comportamento ritenuto più naturale o desiderabile; ad esempio, può verificarsi che una traiettoria che attraversa brevemente una zona in collisione ma si mantiene generalmente distante dagli ostacoli venga considerata più vantaggiosa, in termini di costo, rispetto a una traiettoria che non presenta collisioni ma percorre costantemente percorsi ravvicinati agli ostacoli. Tale comportamento, seppur ottimo dal punto di vista numerico, può risultare poco intuitivo o addirittura inaccettabile in un contesto reale.

L'attuale formulazione della funzione di costo non garantisce che le traiettorie generate siano percepite come naturali o prevedibili da un osservatore umano. Questo aspetto assume particolare rilevanza nei contesti di robotica collaborativa o in ambienti condivisi, dove la leggibilità e l'affidabilità dei movimenti sono fattori chiave per la sicurezza e l'accettazione dell'interazione uomo-robot.

Sebbene CHOMP rappresenti un algoritmo potente e flessibile per la pianificazione di traiettorie, è importante essere consapevoli dei suoi limiti intrinseci, soprattutto in termini di approssimazione geometrica e interpretabilità del comportamento. Tali aspetti devono essere attentamente considerati quando si applica l'algoritmo in scenari reali o sensibili alle interazioni umane.

Per testare i limiti della funzione CHOMP, si procederà a ridurre la distanza tra l'ostacolo superiore e quello inferiore, al fine di generare una simulazione in cui, sebbene nella realtà il manipolatore possa spostare l'oggetto nel punto desiderato evitando collisioni, in simulazione, a causa del modo in cui calcola la traiettoria il CHOMP come spiegato precedentemente, ciò non risulti possibile e venga pertanto imposta una traiettoria che entra in collisione con l'ostacolo. Inizialmente, si analizza il caso in cui la traiettoria calcolata evita gli ostacoli, corrispondente al valore del parametro $b = 0.1$.

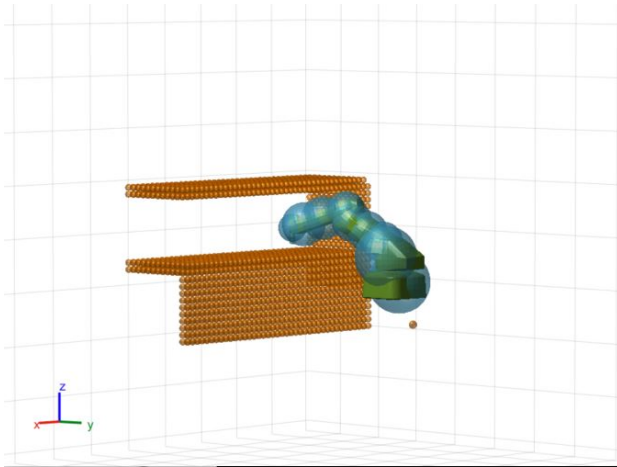


Figura 4.4: Approssimazione CHOMP caso 1

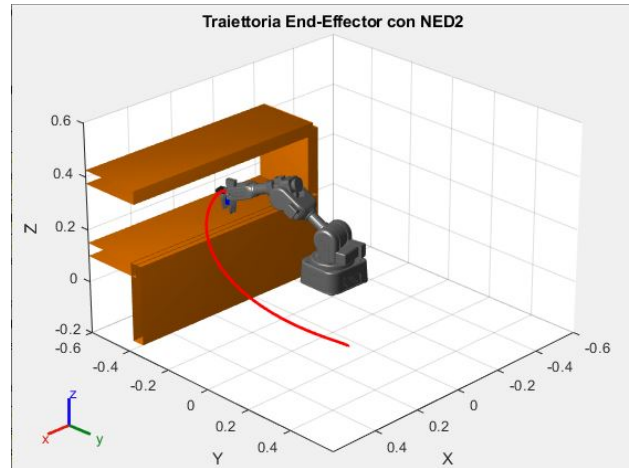


Figura 4.5: Manipolatore caso 1

Si procede ora all'analisi del secondo caso, in cui il parametro b assume il valore di 0.15. Come evidenziato in figura 4.7, il manipolatore reale non entra in collisione con gli ostacoli, a differenza della sua approssimazione sferica. Tale discrepanza comporta che l'algoritmo calcoli una traiettoria forzata che collide con l'ostacolo, nonostante le dimensioni reali del manipolatore gli permetterebbero di evitarlo.

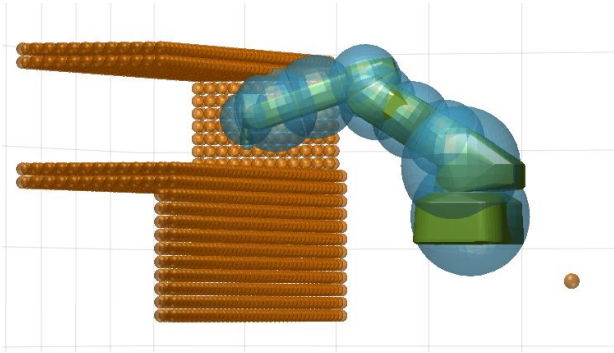


Figura 4.6: Approssimazione CHOMP caso 2

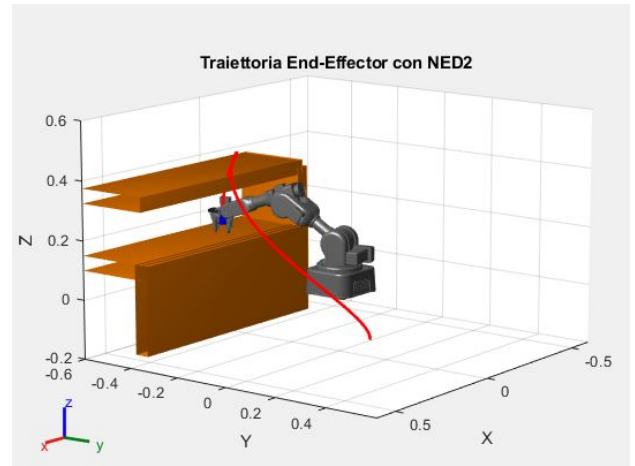


Figura 4.7: Manipolatore caso 2

Capitolo 5

Conclusione e Sviluppi Futuri

Dall'analisi approfondita dei due casi di studio emerge che CHOMP rappresenta un algoritmo efficace per il calcolo di traiettorie caratterizzate da movimenti molto fluidi. Esso approssima l'ambiente di simulazione in maniera sufficientemente accurata e offre un elevato controllo sui parametri dell'algoritmo stesso, tuttavia presenta un limite significativo legato all'approssimazione mediante sfere.

Poiché l'intero ambiente viene approssimato tramite sfere, come già evidenziato, tale approssimazione risulta per eccesso, imponendo quindi un limite fisico alla precisione nel calcolo delle traiettorie ottimali. Di conseguenza, l'algoritmo può generare una traiettoria che, a causa di questa approssimazione, entra in collisione con gli ostacoli, mentre nella realtà il manipolatore, avendo dimensioni minori, non dovrebbe subire collisioni. Questo aspetto può essere migliorato riducendo il volume delle sfere, sfruttando il controllo parametrico messo a disposizione dall'algoritmo.

Pertanto, si può affermare che CHOMP costituisce uno strumento valido per il controllo di manipolatori; tuttavia, in presenza di movimenti di precisione in ambienti ricchi di ostacoli e con spazi di manovra molto ristretti, risulta preferibile adottare soluzioni alternative.

Un possibile sviluppo futuro del lavoro svolto consiste nell'integrazione di tecniche di apprendimento automatico per migliorare l'efficienza e la generalizzazione dell'algoritmo CHOMP. In particolare, MATLAB mette a disposizione un framework per l'addestramento di un *CHOMP optimizer* basato su **Deep Learning** [8], che consente al sistema di apprendere direttamente da dati esperienziali traiettorie ottimali in diversi scenari di ostacoli.

L'approccio proposto prevede la generazione preliminare di un dataset di traiettorie ottimali ottenute in ambienti simulati, le quali vengono successivamente impiegate per l'addestramento di una rete neurale. Tale rete viene istruita a replicare gli aggiornamenti della traiettoria che l'algoritmo CHOMP calcolerebbe tradizionalmente, ma con un vantaggio computazionale sostanziale in fase di esecuzione. I benefici attesi da questa metodologia includono una significativa riduzione dei tempi di calcolo, poiché, una volta completato l'addestramento, la rete è in grado di generare traiettorie ottimali in modo estremamente rapido rispetto ai metodi di ottimizzazione iterativa classici; inoltre, la capacità di generalizzazione del modello consente di adattarsi a configurazioni ambientali inedite, evitando la necessità di ri-ottimizzare la traiettoria da zero per ogni nuovo scenario. Infine, l'approccio garantisce una maggiore robustezza, permettendo la generazione di traiettorie plausibili anche in presenza di rumore o incertezza nei dati sensoriali o nella modellazione dell'ambiente.

Questa estensione basata sul *learning-based CHOMP* si rivela particolarmente promettente per applicazioni in scenari operativi real-time, grazie alla sua capacità di generalizzare traiettorie ottimali in contesti dinamici e complessi. In particolare, essa trova impiego in ambiti quali la robotica collaborativa, dove è fondamentale adattarsi in tempo reale alla presenza di operatori umani e ostacoli in movimento; nella manipolazione robotica in ambienti affollati, caratterizzati da vincoli spaziali stringenti e traiettorie complesse; nonché nella pianificazione adattiva di traiettorie per robot mobili o manipolatori impiegati in contesti di produzione flessibile, in cui le condizioni operative variano fre-

quentemente e richiedono capacità di reazione rapida e affidabile. L'integrazione del Deep Learning con tecniche di pianificazione come CHOMP rappresenta un'importante direzione di ricerca che unisce l'efficienza dell'ottimizzazione classica con la capacità predittiva dei modelli neurali.

Riferimenti

- [1] Matt Zucker et al. «Chomp: Covariant hamiltonian optimization for motion planning». In: *The International journal of robotics research* 32.9-10 (2013), pp. 1164–1193.
- [2] Mrinal Kalakrishnan et al. «STOMP: Stochastic trajectory optimization for motion planning». In: *2011 IEEE international conference on robotics and automation*. IEEE. 2011, pp. 4569–4574.
- [3] John Schulman et al. «Finding locally optimal, collision-free trajectories with sequential convex optimization.» In: *Robotics: science and systems*. Vol. 9. 1. Berlin, Germany. 2013, pp. 1–10.
- [4] Haojian Zhang et al. «Path planning of industrial robot based on improved RRT algorithm in complex environments». In: *IEEE Access* 6 (2018), pp. 53296–53306.
- [5] Nyrio. *Ned2*.
- [6] Nyrio. *Ned2 Custom Gripper*.
- [7] MathWorks. *Robotics system toolbox*.
- [8] MathWorks. *Train Deep-Learning-Based CHOMP Optimizer for Motion Planning*.

Appendice A

Codici Matlab

Innanzitutto è necessario importare il modello del manipolatore in matlab a partire da un file di tipo .sdf dove questo è modellato, rispettivamente:

A.1 ned2 import

File importazione modello sdf del robot

```
1 function [ned2_robot, numJoints] = ned2_import()
2 % NED2_IMPORT Summary of this function goes here
3 % Fabio Mastromarino - Date modified 19/01/2024
4 % This function can import robots that have .stl files associated with the
5 % Unified Robot Description format (URDF) file to describe the visual
6 % geometries of the robot. Each rigid body has an individual visual
7 % geometry specified. The importrobot function parses the URDF file to get
8 % the robot model and visual geometries.
9 % The manipulator's structure would be affected if you edit the file
10 % 'new_ned2.sdf', so avoid doing so.
11 ned2_robot = importrobot("new_ned2.sdf");
12
13 % Joints limit by Niryo Ned Studio [degree] ---      DON'T CHANGE IT
14 limit_low_shoulder = -166.15776058793872;
15 limit_up_shoulder = 166.15776058793872;
16 limit_low_arm = -99.12169855763241;
17 limit_up_arm = 29.220847551671984;
18 limit_low_elbow = -73.6770;
19 limit_up_elbow = 84.22479588423101;
20 limit_low_forearm = -114.01860123103383;
21 limit_up_forearm = 114.01860123103383;
22 limit_low_wrist = -104.27831871380982;
23 limit_up_wrist = 104.45020605234907;
24 limit_low_hand = -139.22874421679006;
25 limit_up_hand = 139.22874421679006;
26
27 % To ensure greater safety, the margin further reduces manipulator limits
28 safety_pos_margin = 0.0;
29
30 % Create the lower limit vector for the six revolute jointsCreate the lower
31 % limit vector for the six revolute joints in radians
32 joints_low_limit_deg = [limit_low_shoulder, limit_low_arm, limit_low_elbow, ...
33     limit_low_forearm, limit_low_wrist, limit_low_hand];
34 joints_low_limit_rad = deg2rad(joints_low_limit_deg + safety_pos_margin);
35
36 % Create the upper limit vector for the six revolute jointsCreate the lower
37 % limit vector for the six revolute joints in radians
```

```

37     joints_up_limit_deg = [limit_up_shoulder, limit_up_arm, limit_up_elbow, ...
38         limit_up_forearm, limit_up_wrist, limit_up_hand];
39
40     % Joints_home represents the initial position of the joints after the
41     % manipulator calibration phase is complete
42     numJoints = 6;
43     joints_home = [0.0, 0.499, -1.251, 0.0, 0.0, 0.002];
44
45     % Checking that ned2.Bodies is a cell_array
46     if iscell(ned2_robot.Bodies)
47         for i = 1:numJoints
48             % For each of the six revolute joints, assign the initial
49             % position as these characteristic isn't directly obtainable
50             % from the file .sdf
51             ned2_robot.Bodies{1,i+1}.Joint.HomePosition = joints_home(i);
52
53             % Assign movement limits to every revolute joint because the
54             % ones in the .sdf file have less stringent safety requirements
55             ned2_robot.Bodies{1,i+1}.Joint.PositionLimits = ...
56                 [joints_low_limit_rad(i), joints_up_limit_rad(i)];
57         end
58     end
59 end

```

Listing A.1: Codice SDF del robot NED2

```

1  <sdf version='1.6'>
2    <world name='default'>
3
4      <gravity>0 0 -9.8</gravity>
5
6      <scene>
7        <ambient>0.4 0.4 0.4 1</ambient>
8        <background>0.7 0.7 0.7 1</background>
9        <shadows>1</shadows>
10     </scene>
11
12     <model name='niryo_robot'>
13
14       <link name='base_link'>
15         <pose frame=''>0 0 0 0 -0 0</pose>
16         <inertial>
17           <pose frame=''>-0.002408 -0.005918 0.051577 0 -0 0</pose>
18           <mass>0.999243</mass>
19           <inertia>
20             <ixx>0.002</ixx>
21             <ixy>0</ixy>
22             <ixz>0</ixz>
23             <iyy>0.002</iyy>
24             <iyz>0</iyz>
25             <izz>0.004</izz>
26           </inertia>
27         </inertial>
28         <collision name='base_link_collision'>
29           <pose frame=''>0 0 0 0 -0 0</pose>
30           <geometry>
31             <mesh>
32               <scale>1 1 1</scale>
33               <uri>collada/base_link.dae</uri>
34             </mesh>
35           </geometry>

```

```

36         <max_contacts>10</max_contacts>
37     </collision>
38     <visual name='base_link_visual'>
39         <pose frame=''>0 0 0 0 -0 0</pose>
40         <geometry>
41             <mesh>
42                 <scale>1 1 1</scale>
43                 <uri>collada/base_link.dae</uri>
44             </mesh>
45         </geometry>
46     </visual>
47     <self_collide>1</self_collide>
48     <enable_wind>0</enable_wind>
49     <kinematic>0</kinematic>
50 </link>
51
52 <joint name='joint_world' type='fixed'>
53     <child>base_link</child>
54     <parent>world</parent>
55 </joint>
56 <!-- The other links and joints follow the same structure -->
57
58 </sdf>

```

Di seguito è riportato il codice matlab per effettuare la simulazione:

A.2 Matlab Code

A.2.1 Init

```

1 %% CARICAMENTO ROBOT e INIZIALIZZAZIONI
2
3 [ned2, numJoints] = ned2_import;
4 ned2.DataFormat = "row";
5 initialguess = ned2.homeConfiguration ;
6 chomp = manipulatorCHOMP(ned2);
7 chomp.CollisionOptions.IgnoreSelfCollision = true ;
8 ik = inverseKinematics("RigidBodyTree", ned2);
9 weight = [0.1 0.1 0 1 1 1];

```

A.2.2 Main

```

1 clear ,clc, close all
2
3 %% Generazione Ambiente
4
5 a = 0.15; % distanza mura- manipolatore
6 b = 0.113 ; %parametro per far scendere muro superiore
7 wallLength = 1; % Lunghezza in X [m]
8 wallHeight = 0.3; % Altezza in Z [m]
9 wallThickness = 0.05; % Spessore in Y [m]
10 wallCenter = [0.25, -0.2-a, 0]; % Posizione centro muro [x y z]
11 wall = collisionBox(wallLength, wallThickness, wallHeight);
12 wall.Pose = trvec2tform(wallCenter); % posizioneMuro(x, y, z)
13 % wall superiore
14 % wall 1
15 wallLength1 =1; % Lunghezza in X [m]

```



```

16 wallHeight1 = 0.05;    % Altezza in Z [m]
17 wallThickness1 = 0.3; % Spessore in Y [m]
18
19 wallCenter1 = [0.25, -0.35-a, (0.5-b)]; % Posizione centro muro [x y z]
20 wall1 = collisionBox(wallLength1, wallThickness1, wallHeight1);
21 wall1.Pose = trvec2tform(wallCenter1); % posizioneMuro(x, y, z)
22
23 %wall 4 --> PAVIMENTO
24
25 wallLength4 = 1;    % Lunghezza in X [m]   scaffale
26 wallHeight4 = 0.05; % Altezza in Z [m]
27 wallThickness4 = 0.3; % Spessore in Y [m]
28 wallCenter4 = [0.25, -0.35-a, 0.125]; % Posizione centro muro [x y z]
29 wall4 = collisionBox(wallLength4, wallThickness4, wallHeight4);
30 wall4.Pose = trvec2tform(wallCenter4); % posizioneMuro(x, y, z)
31
32 % wall 5 --> parete dx
33
34 wallLength5 = 0.05;    % Lunghezza in X [m]
35 wallHeight5 = 0.4;    % Altezza in Z [m]
36 wallThickness5 = 0.3; % Spessore in Y [m]
37 wallCenter5 = [-0.25, -0.35-a, (0.3-b)]; % Posizione centro muro [x y z]
38 wall5 = collisionBox(wallLength5, wallThickness5, wallHeight5);
39 wall5.Pose = trvec2tform(wallCenter5); % posizioneMuro(x, y, z)
40
41
42 %% GENERAZIONE SFERE DI WALL e CILINDRO
43
44 spheresPerMeter = 20; % sfere per metro (risoluzione)%% Calcolo numero di sfere ...
    lungo ogni asse
45 nx = ceil(wallLength * spheresPerMeter);
46 ny = ceil(wallThickness * spheresPerMeter);
47 nz = ceil(wallHeight * spheresPerMeter);
48 sphereRadius = min([wallLength/nx, wallThickness/ny, wallHeight/nz]) / 2; % ...
    Raggio delle sfere
49 sphereRadiusCyl = 0.015 ;
50 cyl = collisionCylinder(sphereRadiusCyl, 0.05);
51 sphObstacles = exampleHelperApproximateCollisionBoxSpheres(wall,sphereRadius);
52 sphObstacles1 = exampleHelperApproximateCollisionBoxSpheres(wall1,sphereRadius);
53 sphObstacles4 = exampleHelperApproximateCollisionBoxSpheres(wall4,sphereRadius);
54 sphObstacles5 = exampleHelperApproximateCollisionBoxSpheres(wall5,sphereRadius);
55 cyl.Pose = targetPose ;
56 sph = exampleHelperApproximateCollisionCylinderSpheres(cyl,sphereRadiusCyl);
57 sphObstacles_total = [sphObstacles sphObstacles1 sphObstacles4 sphObstacles5];
58 sphbstacles = [sphObstacles_total sph];
59 chomp.SphericalObstacles = sphbstacles;
60
61 % Visualizzazione
62 figure;
63 show(chomp);
64 title('Chomp');
65 hold on
66 show(cyl);
67 %%
68 % Traiettorie ottimizzate che evita muro
69
70 startconfig = homeConfiguration(ned2);
71 timepoints = [0 5];
72 timestep = 0.1;
73 placeConfig(1,6) = -placeConfig(1,6)/2 ;
74
75 trajtype = "minjerkpolytraj"; %"minjerkpolytraj";"quinticpolytraj";"interp1" sono ...
    tutte le tipologie di traiettorie

```

```

76
77 chomp.CollisionOptions.CollisionCostWeight = 1;
78 chomp.SmoothnessOptions.SmoothnessCostWeight = 1e-3;
79 chomp.SolverOptions.LearningRate = 5;
80
81 [wptsamples, tsamples] = optimize(chomp,[startconfig; ...
    pick;],timepoints,timestep,InitialTrajectoryFitType=trajtype); %pick
82
83 [wpt1, ts1] = optimize(chomp,[pick; ...
    placeConfig],timepoints,timestep,InitialTrajectoryFitType=trajtype); %place
84
85 figure;
86 show(chomp,wpt1(51,:),NumSamples=10);
87 zlim([-0.5 1.3]);
88 %%
89 % ANIMAZIONE
90
91 % Calcolo la traiettoria dell'end-effector da wptsamples ----> PICK
92 eeTrajectory = zeros(size(wptsamples,1), 3);
93
94 for i = 1:size(wptsamples,1)
95     config = wptsamples(i,:);
96     % Estrai trasformazione dell'end-effector (ultimo link del NED2)
97     tform = getTransform(ned2, config, 'gripper_link');
98     eeTrajectory(i,:) = tform(1:3,4)';
99 end
100 % Calcolo la traiettoria dell'end-effector da wpt1 ----> PLACE
101 eeTrajectory1 = zeros(size(wpt1,1), 3);
102
103 for i = 1:size(wpt1,1)
104     config1 = wpt1(i,:);
105     % Estrai trasformazione dell'end-effector (ultimo link del NED2)
106     tform1 = getTransform(ned2, config1, ned2.BodyNames{end});
107     eeTrajectory1(i,:) = tform1(1:3,4)';
108 end

```

A.2.3 wksp analysis

```

1 %% Workspace analysis
2
3 ned2.Gravity = [0 0 -9.81];
4 ee = "gripper_link" ;
5 [wksp,cfgs] = ...
    generateRobotWorkspace(ned2,{wall,wall1,wall4,wall5},IgnoreSelfCollision="on");
6 mIndexYoshikawa = manipulabilityIndex(ned2,cfgs,IndexType="yoshikawa");
7
8 figure; show(ned2); hold on;
9 show(wall); show(wall1); show(wall4); show(wall5);
10
11 showWorkspaceAnalysis(wksp,mIndexYoshikawa);
12 hold off
13 title("Yoshikawa index");
14 axis auto
15
16 %% PUNTI RAGGIUNGIBILI (trovati post analisi yoshikawa)
17
18 puntoTarget = [0.198064711486166,0.272414567734862,-0.081808696119091];
19 targetPose = trvec2tform(puntoTarget)*axang2tform([0 1 0 pi]);
20
21 [pick, solInfo] = ik("gripper_link", targetPose, weight, initialguess);
22

```

```
23 place = [0.096306675730951,-0.419791864993828,0.221967843573256];
24 targetPlace = trvec2tform(place);
25
26 targetPlace = trvec2tform(place)*axang2tform([0 1 0 pi]);
27 [placeConfig, solInfo1] = ik("gripper_link", targetPlace, weight, pick);
```